Лабораторная работа №2

Параллельные и последовательные области OpenMP

Ответы на вопросы, принципы работы программы и результаты проверки

16 октября 2026 года

Цель работы

Исследовать создание команд нитей, условное распараллеливание, вложенные параллельные области и различие директив single и master. Семь заданий методички реализованы как отдельные опыты в Lab2.cpp.

Как запустить

Откройте ParallelProgramming.sln в Visual Studio с компонентом разработки на C++. Выберите Release x64, соберите и запустите проект. В главном меню введите 2. В меню лабораторной выберите номер задания 1-7 или пункт 8 для последовательного выполнения всех опытов. Пункт 0 возвращает в главное меню, ещё один 0 завершает программу.

OpenMP, C++17 и UTF-8 уже включены в файле проекта. Для повторного запуска на другом Windows-компьютере вручную включать OpenMP не требуется; нужен установленный совместимый набор инструментов C++.

Модель выполнения

Программа начинает работу одной основной нитью. При входе в parallel создаётся команда. Внутри команды породившая нить получает номер 0 и называется мастером. После выхода из области дополнительная команда завершает работу, а породившая нить продолжает последовательный код. Среда выполнения может сохранять рабочие нити в пуле для повторного использования.

```
#pragma omp parallel num_threads(4)
{
    int thread = omp_get_thread_num();
    int count = omp_get_num_threads();
}
```

Каждая нить выполняет весь блок parallel. Эта директива сама не делит обычный цикл; для распределения итераций используется omp for. Номера нитей уникальны только внутри текущей команды.

Состав документа

Страница 2: задания 1 и 2. Страница 3: задания 3 и 4. Страница 4: single и master. Страница 5: задания 6 и 7. Страница 6: устройство программы и синхронизация. Страница 7: фактические результаты и проверка.

Задания 1 и 2: число нитей и опция if

1. Какое максимальное количество нитей позволяет породить система?

В проверенной конфигурации доступны 12 логических процессоров, а omp_get_max_threads() возвращает 12. Эти значения описывают разные вещи: первое - доступные вычислительные ресурсы, второе - текущую настройку размера следующей команды без явной опции num_threads.

omp_get_max_threads() не является абсолютным пределом создания нитей. Опция num_threads может запросить команду большего размера. В опыте showThreadCount() последовательно проверяются 1, 2, 4, 8, 16, 32, 64 и 128 нитей. Фактический размер измеряется внутри каждой области через omp_get_num_threads().

На этом компьютере все запросы до 128 успешно выполнены. Поэтому корректный экспериментальный вывод: система позволяет создать как минимум 128 нитей. Абсолютный предел не установлен, поскольку опыт намеренно ограничен этим диапазоном и не доводит процесс до исчерпания ресурсов.

Число созданных программных нитей может превышать число логических процессоров. Тогда ОС поочерёдно назначает их на процессоры. Это называется избыточным созданием нитей и обычно увеличивает расходы переключения, а не ускоряет вычисления. Для реальной обработки данных разумная отправная точка - число доступных логических процессоров.

2. В каких случаях нужна опция if у parallel?

Создание команды, распределение работы и синхронизация занимают время. Для маленькой задачи эти расходы могут превысить выигрыш от параллельного выполнения. Опция if позволяет использовать несколько нитей только при выполнении заданного условия.

```
#pragma omp parallel if(size >= 100000) num_threads(4)
{
    // При ложном условии блок выполняет одна нить
}
```

Ложное условие не пропускает блок: создаётся сериализованная область с одной нитью. Это отличается от обычного if в C++, где тело вообще не выполняется, если условие ложно.

В showConditionalParallel() вычисляется сумма целых чисел от 1 до size. При size = 10 область выполняется одной нитью, при size = 1 000 000 - четырьмя. Итерации распределяет omp for, а reduction(+:sum) создаёт частные суммы и объединяет их после области.

Результат сравнивается с формулой size × (size + 1) / 2. Для 10 ожидается 55, для миллиона - 500 000 500 000. Тип long long нужен, потому что второй результат не помещается в обычный 32-битный int. Порог 100 000 выбран для демонстрации и не объявляется измеренным оптимумом.

Задания 3 и 4: процессоры и вложенность

3. Сколько процессоров доступно и как назначить по одной нити?

omp_get_num_procs() возвращает количество логических процессоров, доступных процессу на момент вызова. Логический процессор не обязательно равен физическому ядру: одно ядро с SMT может предоставлять несколько аппаратных потоков.

```
int processors = omp_get_num_procs();

#pragma omp parallel num_threads(processors)
{
    // По одному участнику команды на логический процессор
}
```

В showAllProcessors() запрошена команда из 12 нитей. Каждая выводит номер и размер команды. Это выполняет учебное распределение «одна нить на логический процессор». Жёсткая привязка конкретной нити к определённому ядру отдельно не задаётся: фактическое размещение выполняют ОС и среда OpenMP.

4. Как получить восемь ветвей на трёх уровнях?

Функция nestedRegion() создаёт команду из двух нитей. Каждая нить вызывает её для следующего уровня, пока level не достигнет 3. При разрешённой вложенности получается 2 × 2 × 2 = 8 выполнений блока самого глубокого уровня.

Путь вида 0.1.0 показывает номер нити на каждом уровне: сначала нить 0 внешней команды, затем нить 1 её дочерней команды, затем нить 0 команды третьего уровня. Один локальный номер не позволил бы различать ветви, потому что каждая команда нумерует участников заново.

Режим	Уровень 1	Уровень 2	Уровень 3
Вложенность отключена	2 ветви	2 ветви	2 ветви
Вложенность включена	2 ветви	4 ветви	8 ветвей

При omp_set_nested(0) внутренние области не пропускаются. Они выполняются сериализованными командами размера 1. Поэтому две внешние нити проходят по одной ветви до третьего уровня: итоговый счётчик равен 2.

При omp_set_nested(1) внутренние команды активны и счётчик равен 8. Сумма 2 + 4 + 8 не является числом независимо работающих исполнителей: родительская нить сама участвует в дочерней команде. Восемь ветвей не означают гарантированное одновременное выполнение восьми инструкций - это зависит от доступных процессоров и планировщика.

leafCount - общий счётчик завершений третьего уровня. Он увеличивается с omp atomic, чтобы одновременные изменения не терялись. Настройка вложенности после опыта возвращается к исходному значению.

Задание 5: single и master

5. Чем отличаются эти директивы?

Обе директивы позволяют выполнить выделенный блок одним участником команды, однако правила выбора исполнителя и ожидания отличаются. Они распределяют работу внутри существующей команды и не создают дополнительные нити.

Свойство	single	master
Исполнитель	Любая одна нить текущей команды	Только нить 0 текущей команды
Неявный барьер на выходе	Есть, если не указан nowait	Нет
Выбор повторного исполнителя	Может изменяться между выполнениями	Всегда локальный номер 0
Типичное применение	Однократная работа без привязки к номеру	Работа, назначенная мастеру

Почему после single по умолчанию ждут все?

Одна нить могла подготовить общие данные, которые затем нужны остальным. Неявный барьер на выходе не позволяет другим нитям использовать результат раньше завершения подготовки. У single нет барьера на входе, поэтому разные нити могут подойти к нему в разное время.

single nowait удаляет выходной барьер. Это удобно, если следующие действия независимы от результата выделенного блока. Если же другие нити сразу читают изменённые общие данные, потребуется отдельная синхронизация.

Почему master не обеспечивает ожидание?

Остальные нити пропускают master и сразу продолжают свой код. Поэтому при подготовке общих данных мастером необходим явный barrier перед их использованием. Барьер должны встретить все нити команды, а не только мастер.

```
#pragma omp parallel num_threads(4)
{
#pragma omp master
    {
        prepareData();
    }

#pragma omp barrier
    useData();
}
```

Что показывает реализованный опыт?

showSingleAndMaster() выполняет оба блока и считает, сколько раз они запускались. Ожидается singleCount = 1 и masterCount = 1. После master установлен явный barrier; затем все нити выводят завершение этапов. Сообщение single всегда напечатано раньше master, потому что в single действует выходной барьер.

В контрольном запуске single выполнила нить 2, master - нить 0. При другом запуске single может выбрать иного участника; совпадение номеров не делает конструкции эквивалентными.

Задания 6 и 7: кто исполняет блок?

6. Может ли нить-мастер выполнить single?

Да. Нить 0 является полноправным участником команды и может быть выбрана для single. Стандарт не исключает её и не назначает ей особого приоритета при выборе исполнителя.

showMasterInSingle() сначала создаёт команду из одной нити. В ней единственный исполнитель имеет номер 0, поэтому single гарантированно выполняет мастер. Это сериализованная параллельная область: она подтверждает допустимость такого исполнителя, но не наличие одновременной работы нескольких нитей.

Затем опыт повторяется с командой из четырёх нитей. Теперь single может выполнить любой из номеров 0, 1, 2 или 3. Порядок исполнения определяется средой; даже если на одном компьютере постоянно выбирается 0, переносить эту закономерность в логику алгоритма нельзя.

В сохранённом выводе команды из нескольких нитей выбран номер 0. Это результат конкретного запуска, а не требование OpenMP.

7. Может ли нить с номером 1 выполнить master?

Нет, если речь идёт о локальном номере в текущей команде. master по определению исполняет только участник с номером 0.

```
#pragma omp parallel num_threads(4)
{
    int value = 1;

#pragma omp master
    {
        value = 2;
    }

    // Нить 0 имеет value = 2, остальные - value = 1
}
```

В showMasterOnly() value объявлена внутри parallel и поэтому имеет отдельный экземпляр в каждой нити. Все копии сначала равны 1. Мастер меняет только собственную копию на 2. В результате нить 0 выводит 2, а нити 1, 2 и 3 выводят 1.

После master барьер не требуется для правильности этого опыта: разные нити не читают и не меняют чужую value. Критическая секция вокруг cout нужна только для цельных строк вывода и не делает value общей.

Особенность вложенных команд

Нить с номером 1 во внешней команде может породить внутреннюю команду и стать в ней мастером. Но её локальный номер внутри дочерней команды будет 0. Поэтому это не исключение из правила: проверять номер нужно относительно той команды, в которой находится master.

Устройство программы и работа с памятью

Организация кода

runLab2() выводит меню и через switch вызывает отдельную функцию для каждого задания. Вспомогательные функции находятся в безымянном namespace: они доступны только внутри Lab2.cpp. Заголовок Lab2.h по-прежнему содержит единственную внешнюю функцию runLab2().

Функция	Назначение
showThreadCount()	Проверка заданных размеров команды до 128
showConditionalParallel()	Опция if и проверка суммы через reduction
showAllProcessors()	Команда по числу доступных процессоров
showNestedRegions() / nestedRegion()	Сравнение двух режимов вложенности
showSingleAndMaster()	Исполнители и однократность single / master
showMasterInSingle()	Допустимость мастера как исполнителя single
showMasterOnly()	Изменение локальной копии только у мастера

Общие и локальные данные

actual, leafCount и счётчики выполнений создаются до параллельных областей и доступны команде как общие. thread, path и value объявлены внутри областей и принадлежат конкретным нитям. firstprivate(level, parent) копирует параметры рекурсивного вызова каждому участнику вложенной команды.

reduction(+:sum) создаёт у каждой нити собственную сумму с нейтральным значением 0. После завершения области частные суммы объединяются. Поэтому в цикле нет сериализации каждого сложения через critical.

Для чего используются critical и atomic?

critical(outputLab2) защищает цепочку операций вывода: одна строка полностью печатается одной нитью, прежде чем другая войдёт в такую же именованную секцию. Порядок строк при этом остаётся произвольным. atomic защищает только увеличение leafCount и подходит для этой простой общей операции.

Восстановление состояния и ввод

Перед опытами runLab2() сохраняет omp_get_dynamic() и отключает автоматическое изменение размера команды. Это делает запросы num_threads воспроизводимыми. После выхода исходное значение восстанавливается. showNestedRegions() аналогично сохраняет и восстанавливает разрешение вложенности.

При текстовом вводе вместо номера поток очищается, ошибочная строка пропускается и меню показывается повторно. Завершение входного потока обрабатывается как выход из меню. В главном меню добавлена такая же проверка EOF, чтобы программа не повторяла сообщение об ошибке бесконечно при запуске через перенаправленный ввод.

Фактические результаты и проверка

Проверка выполнена 16 октября 2026 года на текущем Windows-компьютере в Visual Studio 2022. Сборки Release x64 и Debug x64 завершены без ошибок. Приведённые числа относятся к этой системе и могут отличаться на другом устройстве.

Создание команд

Запрошено	Создано	Запрошено	Создано
1	1	16	16
2	2	32	32
4	4	64	64
8	8	128	128

Доступно логических процессоров: 12. Настройка команды по умолчанию: 12. Верхняя проверенная граница опыта равна 128; это не утверждение об абсолютном пределе среды.

Проверка поведения

Опыт	Результат
if, size = 10	1 нить, сумма 55, эталон совпал
if, size = 1 000 000	4 нити, сумма 500 000 500 000, эталон совпал
Вложенность отключена / включена	2 / 8 выполнений третьего уровня
single и master	По одному выполнению каждого блока
Локальное значение в master	У нити 0: 2; у нитей 1-3: 1

Что дополнительно проверено

Все семь пунктов запускались по отдельности и через «Выполнить все задания». Проверены ошибочный номер, текстовый ввод, завершение потока и повторный вход в лабораторную. При OMP_NUM_THREADS=3 и OMP_DYNAMIC=TRUE опыт с if всё равно корректно создаёт явно запрошенную команду из четырёх нитей.

Полный вывод контрольного прогона сохранён в output/lab2_run.txt. Для повторения выберите в главном меню 2, затем 8, затем дважды 0. Порядок строк разных нитей может меняться; проверять нужно размеры команд, суммы и итоговые счётчики, а не точное расположение строк.

Итог

Эксперименты подтверждают создание команд по num_threads, сериализацию области при ложном if, отличие разрешённой вложенности от запрещённой и разницу выбора исполнителя у single и master. Все семь пунктов задания имеют отдельную демонстрацию, а ответы вынесены в этот документ.

Источник задания: предоставленный файл «ППЛабораторная_работа_2.pdf», раздел «Задания», страницы 10-11. Примеры и результаты в документе относятся к обновлённому Lab2.cpp.